

비트연산으로 조합풀기, BFS 경로 추적하기

👤 소유자	👤 암암코딩
🏷️ 태그	
🕒 생성 일시	@2024년 8월 15일 오전 11:36

기본 비트연산 방법

연산자	연산자의 기능
&	비트단위로 AND 연산을 한다.
	비트단위로 OR 연산을 한다.
^	비트단위로 XOR 연산을 한다.
~	단항 연산자로서 피연산자의 모든 비트를 반전시킨다.
<<	피연산자의 비트 열을 왼쪽으로 이동시킨다.
>>	피연산자의 비트 열을 오른쪽으로 이동시킨다.

한 숫자를 2진수로 표현 했을 때

0번 bit가 0인지 1인지 구분을 해보자

```
int main()
{

    int d = 0x6;
    int result = 0;

    result = d & 0x1;

    if (result > 0)
    {

    }

}
```

```
    return 0;  
}
```

한 수자를 2진수로 표현 했을 때 1번 비트가 0인지 아닌지 구분하고 싶다면 다음과 같이 생각해 볼수 있다.

```
int d = 0x6;  
int result = 0;  
  
result = (d >> 1) & 0x1;  
  
if (result > 0)  
{  
    int a = 0;  
}
```

N번 비트가 0인지 아닌지 비교하고 싶다면?

```
int d = 0x6;  
int result = 0;  
  
result = (d >> n) & 0x1;  
  
if (result > 0)  
{  
    int a = 0;  
}
```

비트연산자를 이용해서 갯수를 count 해보자

0~15사이 숫자를 하나 입력받고

2진수 변환했을 때 1이 몇개인지 세어보자

for문돌려서 세어보고

최대 입력값인 15는 2진수로 0000 1111 8자리 숫자

10진수나 16진수나 2진수나 컴퓨터는 다 똑같은 숫자로 인식을 한다.

따로 진수간에 변환해서 작업할 필요가 없다.

```
int main()
{
    int d;
    int result;
    int count;

    std::cin >> d;

    count = 0;

    for (size_t i = 0; i < 8; i++)
    {
        result = (d >> i) & 0x1;

        if (result > 0)
        {
            count++;
        }
    }

    std::cout << count << std::endl;

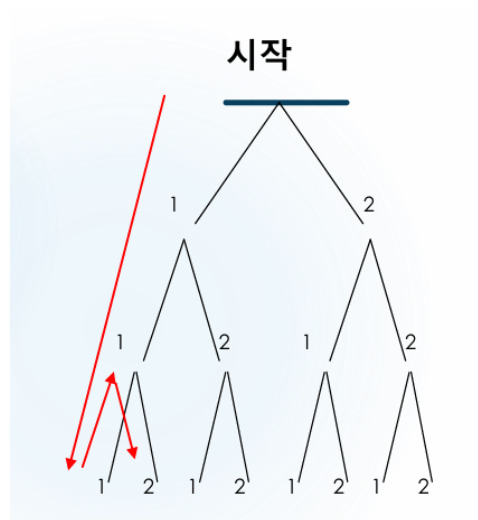
    return 0;
}
```

비트연산자 사용하실때 주의할점이 있습니다.

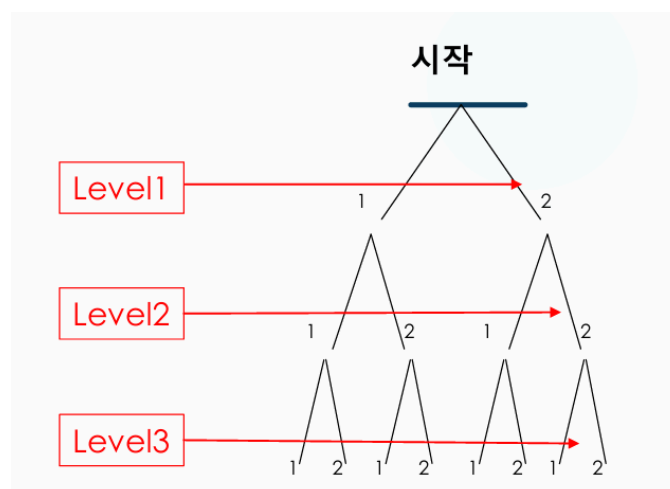
1. 연산자 우선순위 잘 체크하고 사용하자 (괄호를 사용하자)

DFS vs BFS

dfs 깊이 우선탐색으로 탐색순서가 한쪽으로 가장 밑에까지 내려가면서 하나 씩 탐색하는 방법이다.



bfs는 너비 우선 탐색으로 level별로 오른쪽으로 가면서 탐색하는 방법이었다.



BFS 로 풀어야 하는 문제

모든 DFS 문제는 BFS로 풀 수 있다.

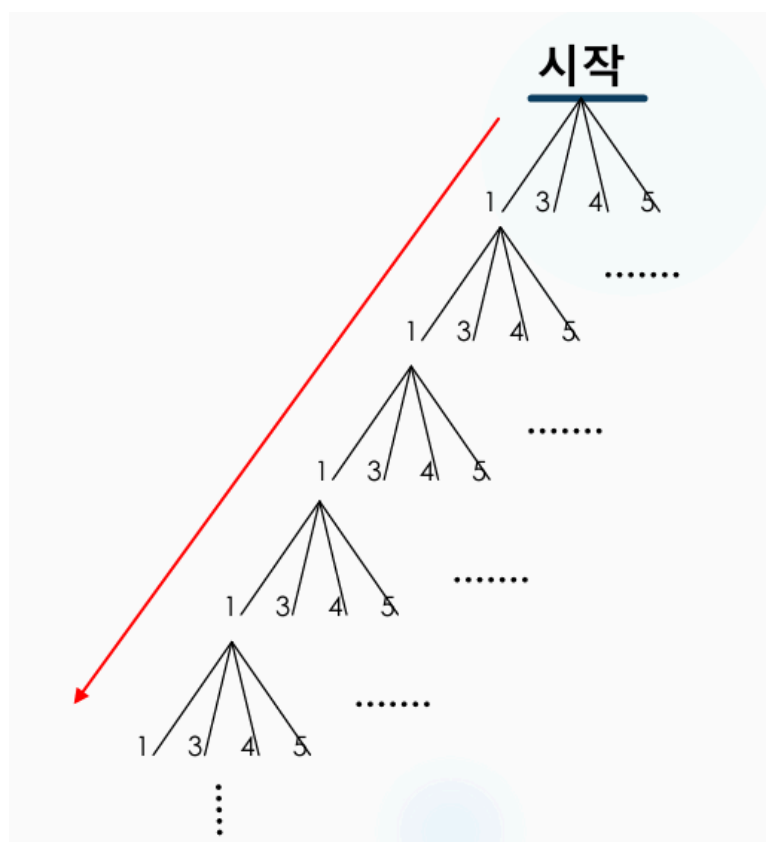
BFS로 풀어야 답이 빨리 나오는 문제를 알아보자

네 종류의 숫자카드를 여러 개 선택해서 합이 12가 나오도록 한다.

최소 몇장의 카드를 선택 해야 12를 만들 수있나.

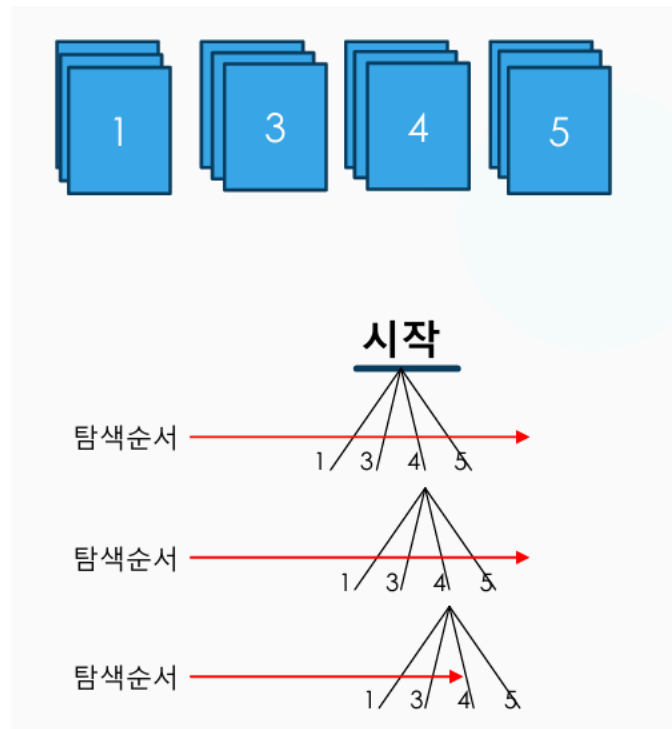


DFS로 풀게되면 맨 아래쪽으로 끝까지 들어갔다가 나와야 한다. (오래걸림)



4장 전부를 뽑지않고 해당 결과를 만들수 있는 문제라면 BFS를 활용하면 된다.

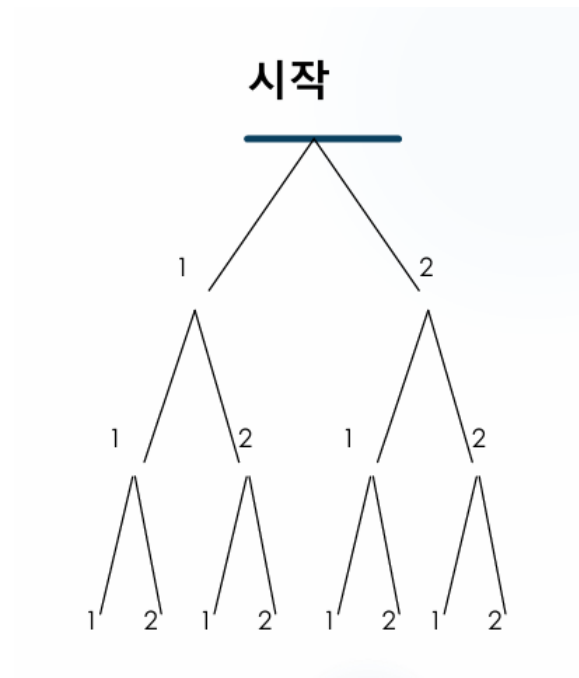
BFS로 풀게되면 level3을 탐색 할 때 합이 12인 경우를 찾을 수 있다.
DFS보다 빠르게 답을 구할수 있다.



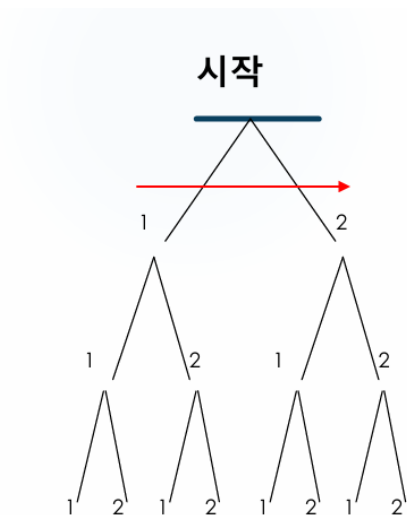
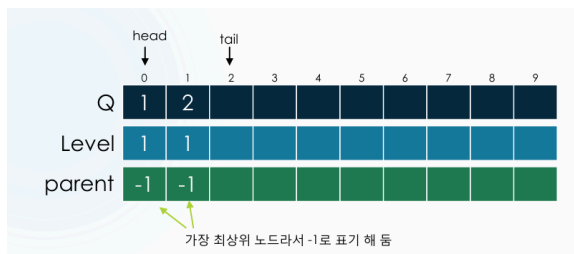
경우의수를 BFS를 이용해서 풀어보자

권투선수가 원(1) 또는 투(2) 펀치를 이용해서 공격을 한다.

이권투선수가 세번의 펀치를 날릴때 모든 경우의수를 BFS 를 이용해서 출력해보자.

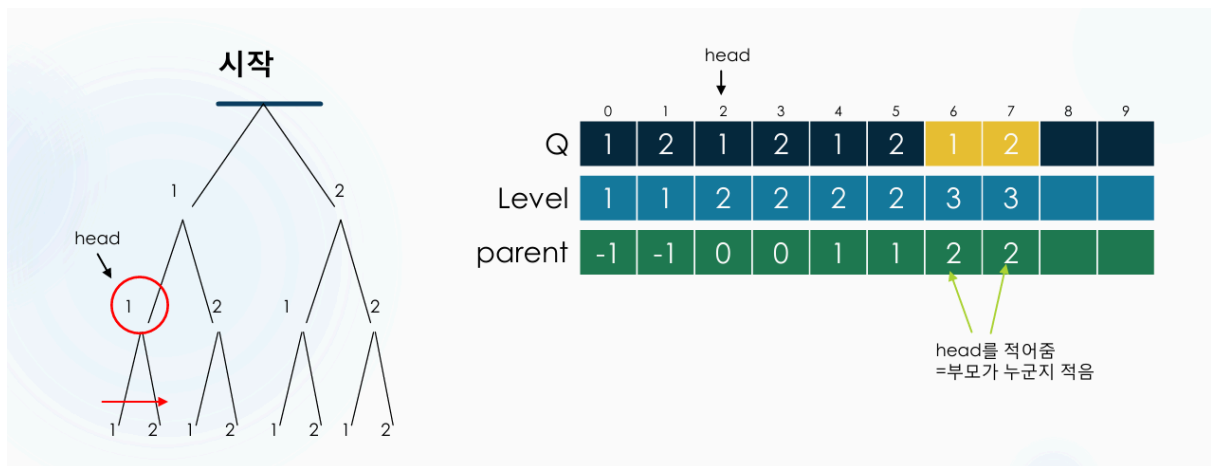
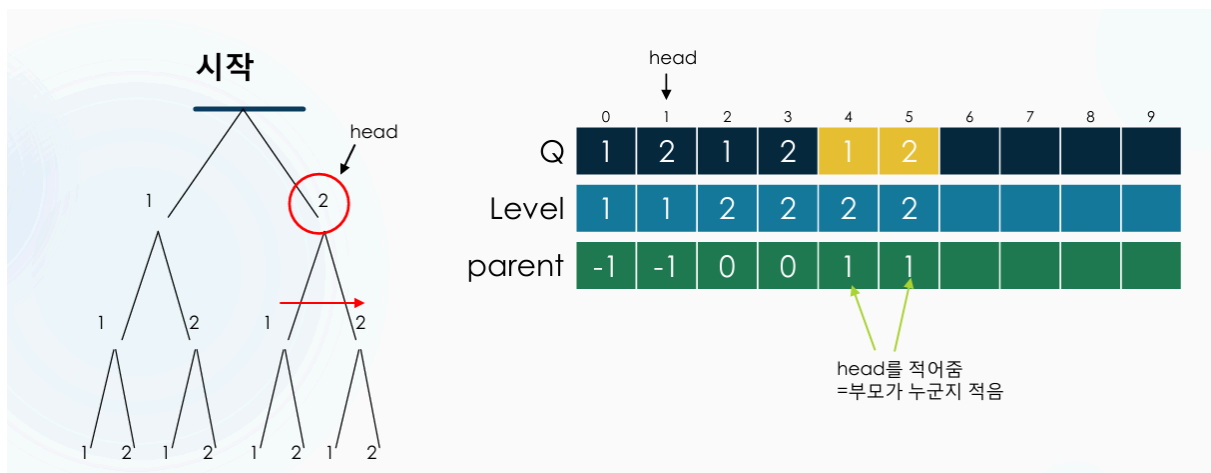
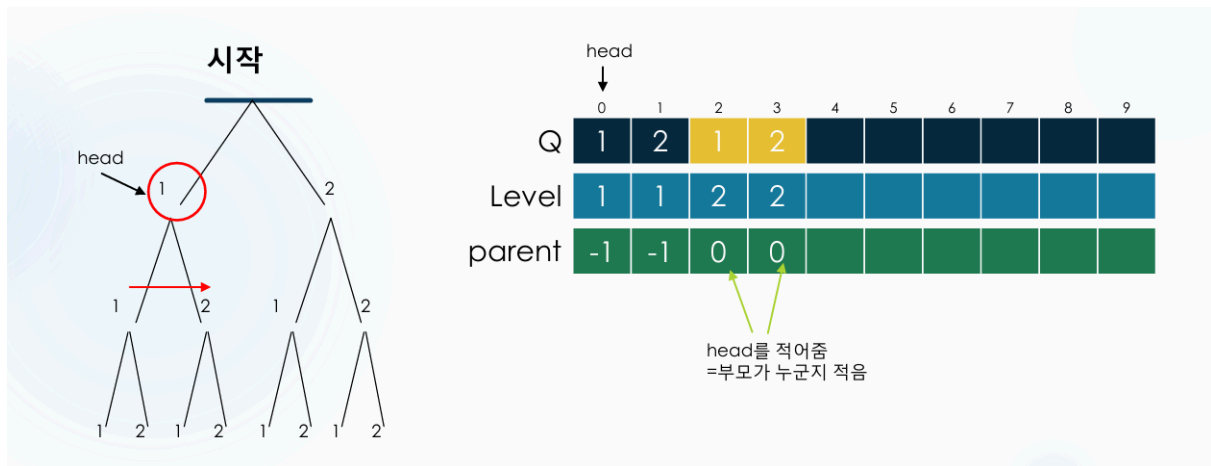


먼저 level 1에 있는 내용을 써두고 시작한다.



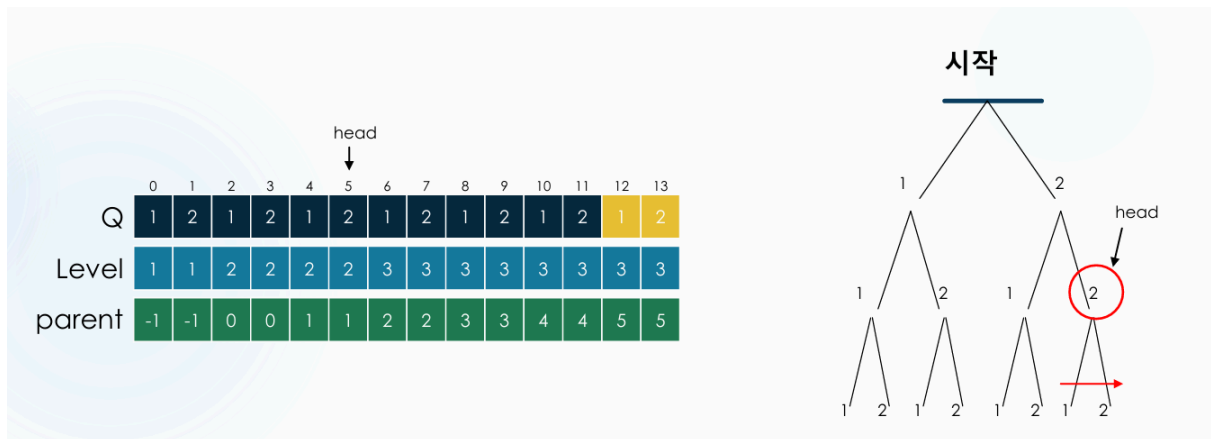
이제 자식노드들에 대한 정보를 queue 프론트 노드에 접근해서 채워준다.

다채웠으면 queue에서 프론트를 빼주고 pop 하면 그다음 프론트 (2번째 노드) 같은 작업을 반복한다.



이렇게 반복 하다 보면 아래와 같이 queue에 채워질꺼다 .

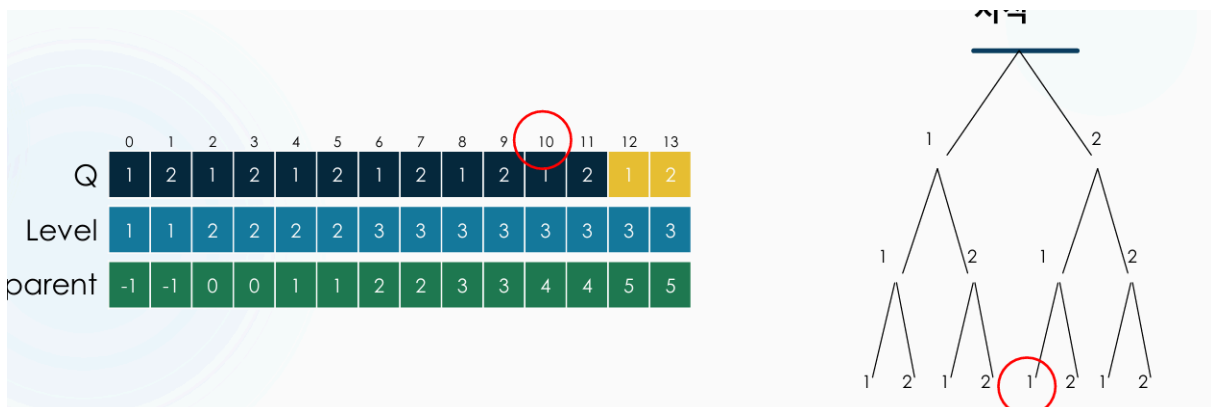
우리는 세번의 펀치기때문에 level 3까지만 반복하면 된다.



각 노드별로 저장되어있는 queue가 완성된다.

노드가 총 14개 이기 때문에 14칸의 queue가 완성 됐을 것이다.

예를 들어서 10번인덱스의 queue정보는 트리에서 빨간색 위치를 의미한다.



BFS queue 채우기 코드

```
#include <iostream>
#include <queue>

int queue[30] = { 1, 2 };
int level[30] = { 1, 1 };
int parent[30] = { -1, -1 };

int head = 0;
```

```

int tail = 2;

int main()
{
    while (true)
    {
        for (size_t i = 0; i < 2; i++)
        {
            queue[tail] = i + 1; // 원 투
            level[tail] = level[head] + 1;
            parent[tail] = head;
            tail++;
        }

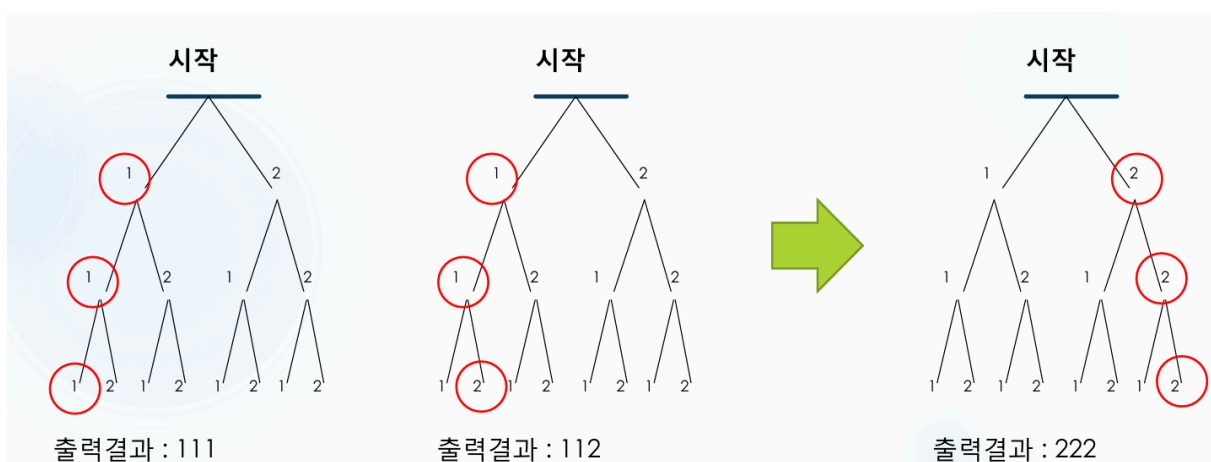
        head++;

        if (level[head] == 3)
            break;
    }

    return 0;
}

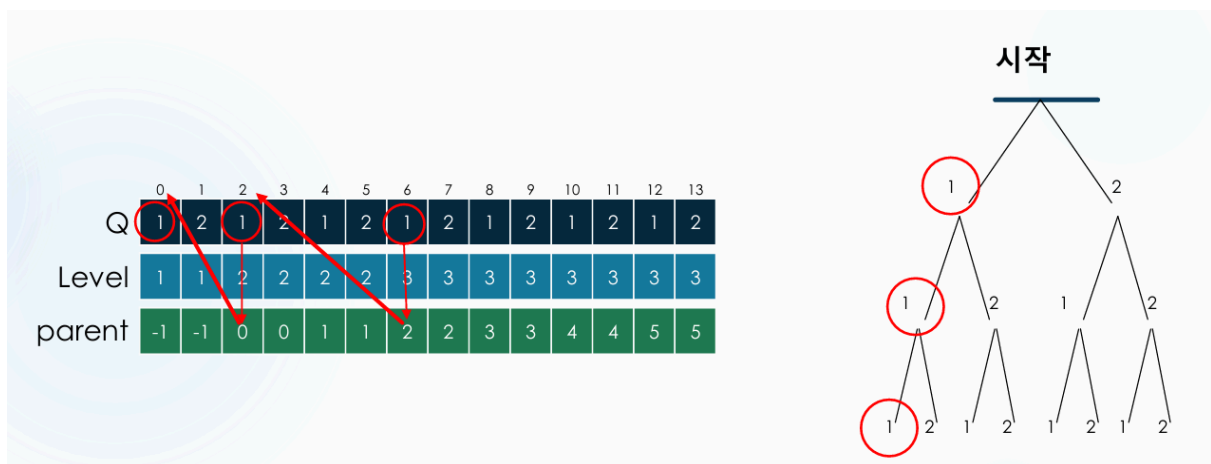
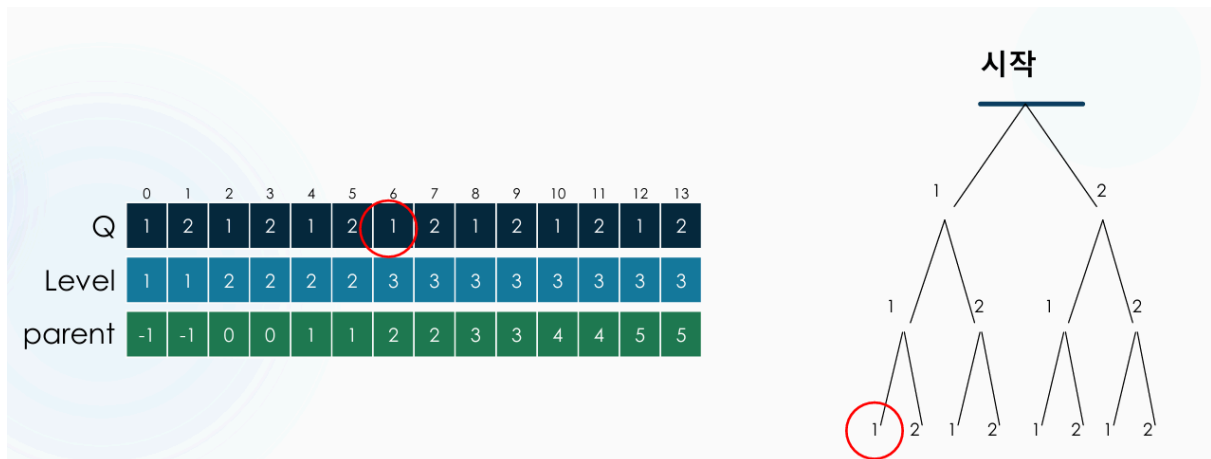
```

결과를 출력하면 자식노드에서 부모노드를 시작지점까지 출력해나가면 된다.



BFS는 자식 노드부터 거꾸로 부모노드로 추적해 나가야한다.

parent 배열을 보면 현재 노드의 부모노드 위치를 알수 있다. (queue 에 부모위치의 인덱스 값)



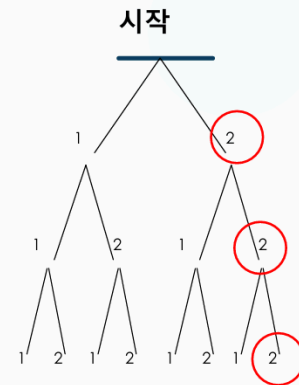
따라서 반복문을 돌면서 추적해가면서 출력하면 된다.

```

for (자식노드 1 ~ 8)
{
    추적하며 출력하기(자식노드번호);
}

```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Q	1	2	1	2	1	2	1	2	1	2	1	2	1	2
Level	1	1	2	2	2	3	3	3	3	3	3	3	3	3
parent	-1	-1	0	0	1	1	2	2	3	3	4	4	5	5



```

#include <iostream>
#include <queue>

int queue[30] = { 1, 2 };
int level[30] = { 1, 1 };
int parent[30] = { -1, -1 };

int head = 0;
int tail = 2;

void print(int idx)
{
    while (true)
    {
        if (idx == -1)
            break;

        std::cout << queue[idx] << " ";
        idx = parent[idx];
    }
}

int main()
{

```

```

while (true)
{
    for (size_t i = 0; i < 2; i++)
    {
        queue[tail] = i + 1; // 원 투
        level[tail] = level[head] + 1;
        parent[tail] = head;
        tail++;
    }

    head++;

    if (level[head] == 3)
        break;
}

for (size_t i = head; i < tail; i++)
{
    std::cout << std::endl;
    print(i);
}

return 0;
}

```

연습문제



문제 1번



시골집에 벌들이 너무 많이 증식하여 골치를 앓고 있습니다.

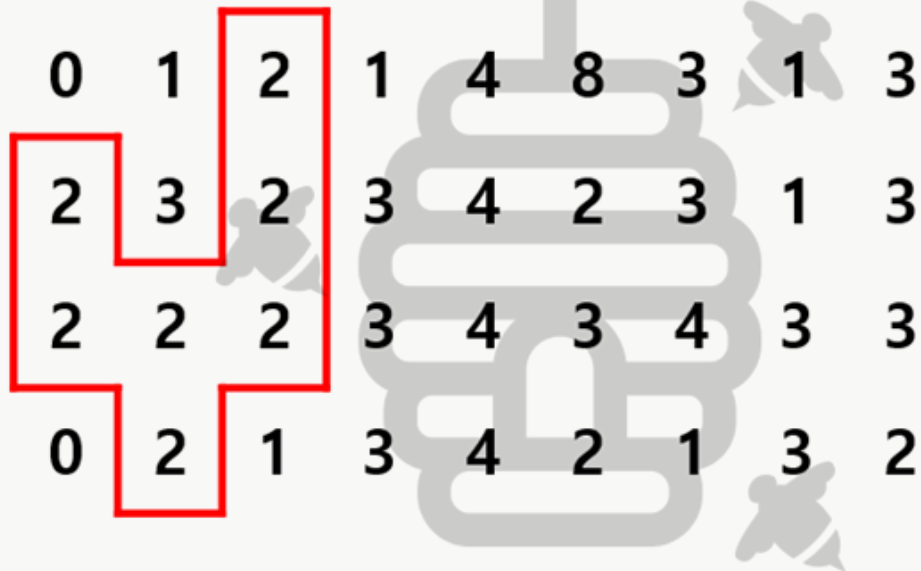
벌들을 죽여도 소용이 없어 이장님은 여왕벌의 알집을 찾아 없애려고 합니다.

입력으로 4 x 9 벌집이 입력 됩니다. 벌집안의 숫자는 벌들의 신호입니다.

어느 숫자가 알집을 나타내는 지는 이장님은 알 수 없지만,

같은 숫자가 가장 많이 붙어 있는 지역이 알집인 것을 알아 내셨습니다.

[알집은 상, 하, 좌, 우로만 붙어 있습니다.]



[입력]

4 X 9 맵이 입력 됩니다.

[출력]

연속으로 붙어있는 숫자가 가장 많을때, 개수와 집의 값을 곱한 값을 출력하세요.

입력 예제

```
0 1 2 1 4 8 3 1 3
2 3 2 3 4 2 3 1 3
2 2 2 3 4 3 4 3 3
0 2 1 3 4 2 1 3 2
```

출력 결과

14



문제 2번

현재 채널에서 목표 채널로 이동할 수 있는 최소 버튼 클릭 회수를 출력해주세요.



리모콘에는 채널을 변경할 수 있는 4개의 버튼이 있습니다.



목표채널로 이동하기 위해서 최소 몇 번의 버튼을 눌러야 하는지 알려주는 프로그램을 BFS를 이용하여 작성 해 주세요.

[입력]

현재 채널과 목표 채널을 입력해주세요.

[출력]

현재 채널에서 목표 채널까지 이동하는데 누르는 버튼의 최소 횟수를 출력해주세요.

[예시]

현재 3번 채널이고, 9번 채널로 이동하기 위해서는

버튼을 순서대로 [+1], [x2], [+1] 3번 누르면, 최소의 클릭 횟수로 목표 채널로 이동할 수 있습니다.

입력 예제

3

9

출력 결과

3

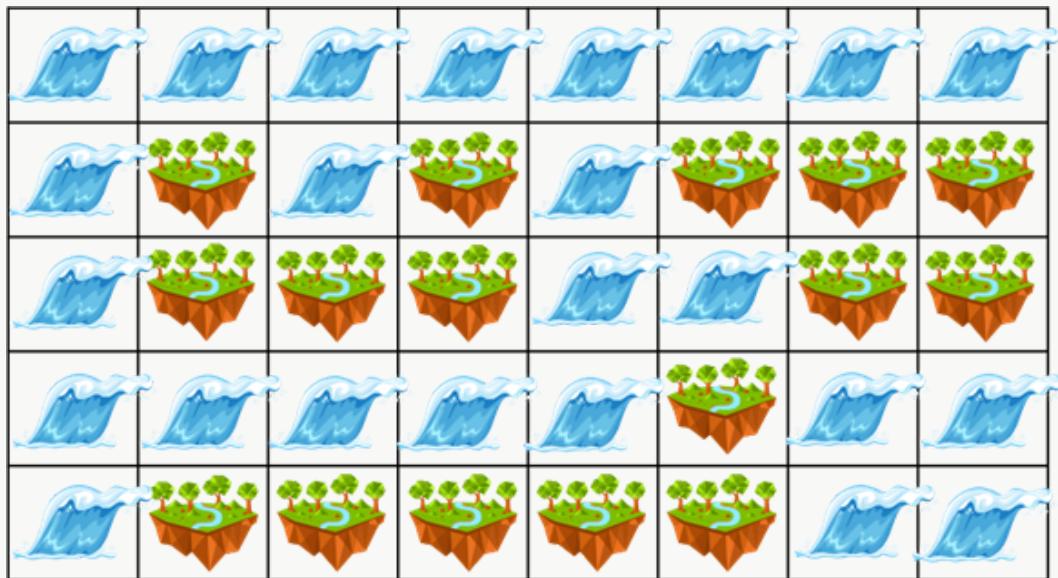


문제 3번

주어지는 지도에 몇개의 섬이 존재하는지 찾아내야 합니다.

지도는 5 x 8 크기로 주어지며, 하나의 섬은 상, 하, 좌, 우로 이어져있습니다.

예를들어, 아래 지도와 같은 경우 이어져 있는 섬의 개수는 총 3개 입니다.



[입력]

5 × 8 크기의 지도 정보가 입력 됩니다.

섬은 1, 바다는 0 으로 입력 됩니다.

[출력]

이어져 있는 각 섬의 총 개수를 출력 하세요.

입력 예제

0 0 0 0 0 0 0 0

0 1 0 1 0 1 1 1

0 1 1 1 0 0 1 1

0 0 0 0 0 1 0 0

0 1 1 1 1 1 0 0

출력 결과



문제 4번

해물새우파전의 맛은 타이거 새우와 오징어의 배치가 큰 역할을 합니다.

[제공에 필요한 규칙]

1. 타이거 새우의 위치는 각각 상, 하, 좌, 우 기준으로 세 칸 이상 떨어져 있어야 합니다.
2. 오징어의 위치는 각각 상, 하, 좌, 우 기준으로 네 칸 이상 떨어져 있어야 합니다.

위의 규칙을 해물 파전마다 확인하고,

완성된 해물 파전을 확인하여 손님에게 나갈 수 있는지 출력해주세요.

[입력]

해물 새우 파전의 정보가 입력 됩니다.

1 은 새우를 의미하고, 2 는 오징어를 의미합니다.

[출력]

손님에게 제공할 수 있으면 "pass", 할 수 없다면 "fail" 을 출력 해주세요.

입력 예제

0001200

0200000

0010012

0000000

1001000

0200010

0010200

출력 결과

pass



문제 5번

빙하 왕국의 엘사와 안나가 빙하 평원에서 길을 잃었습니다.

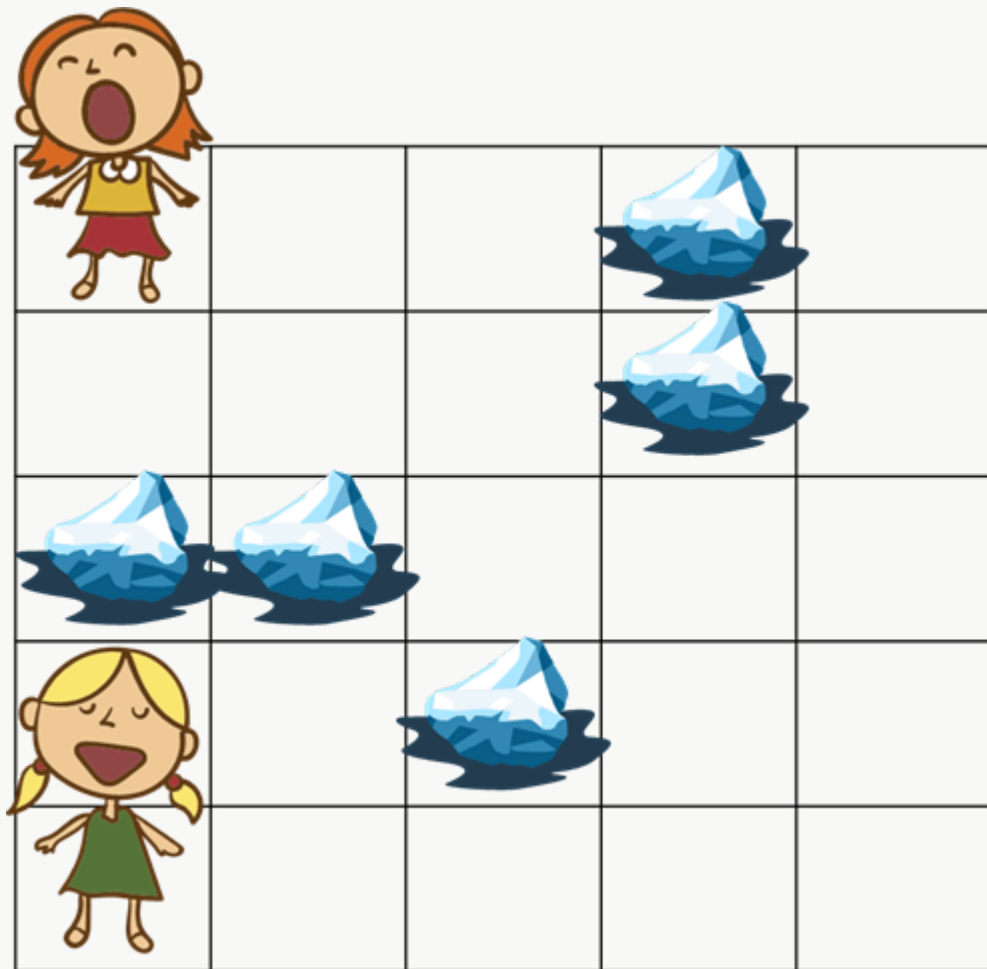
빙하를 피해 엘사와 안나가 가장 빠르게 만날 수 있는 이동시간을 계산해주세요.

엘사와 안나는 1초 마다 네 방향으로 각각 상, 하, 좌, 우로 움직일 수 있습니다.

또한, 엘사와 안나는 제자리에 멈춰있는 것도 가능합니다.

빙하 평원에서 빙하가 있는 위치로는 안나와 엘사가 이동할 수 없습니다.

빙하의 위치는 아래 그림과 같습니다.



[입력]

엘사의 위치 y, x 가 입력 된 후에, 안나의 위치 y, x 가 입력 됩니다.

[출력]

두 사람이 같은 위치에서 만나게 될 때, 가장 최소 이동시간을 출력 해주세요.

입력 예제

0 0

4 0

출력 결과

5



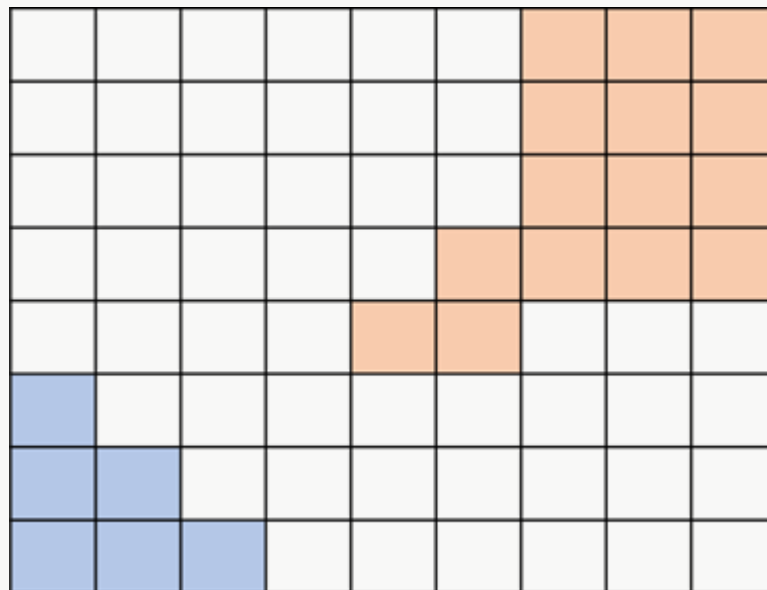
문제 6번

천지창조는 신이 인간을 흙으로 만들고 생명을 불어 넣는 것을 표현한 그림입니다.

아래 그림에서 이 두사람은 각각 **오른쪽 위** 와 **왼쪽 아래 모서리에** 배치되어 있습니다.



아래는 8 x 9 Size의 pixel로 표현 한 예제입니다.



[힌트]

1. 한 사람을 기준으로 BFS돌려 한사람의 좌표를 큐에 모두 넣습니다.
2. 다시 BFS를 돌려 가장 가까운 땅의 거리를 찾습니다.

[입력]

8 × 9 크기의 그림 정보를 입력 해주세요.

[출력]

두 사람을 연결하기 위한 최소 거리를 출력해주세요.

- 상, 하, 좌, 우 방향으로만 연결될 수 있습니다.

입력 예제

____###

____###

____###

____####

____##_

#_____

##______

###_____

출력 결과

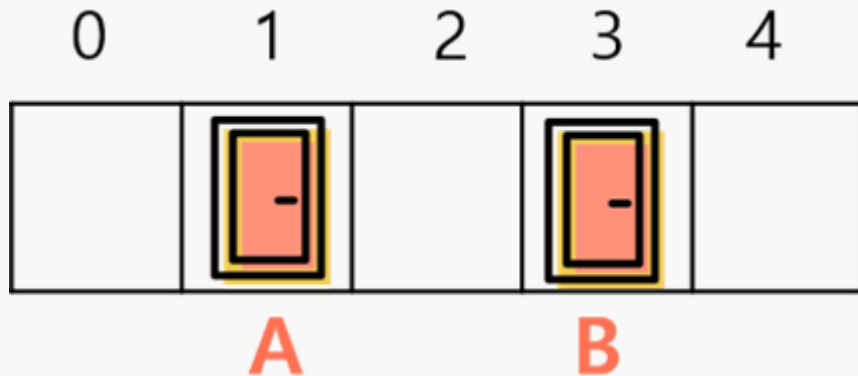
4



문제 7번

5칸의 입구와 2개의 미닫이 문 (A, B)가 있습니다.

손님이 입장하는 번호 칸에 미닫이 문이 없어야 입장이 가능합니다.



예로들면 현재 미닫이문은 1번과 3번에 위치하기 때문에, 손님들은 지금 1번과 3번으로 입장할 수 없습니다.

미닫이 문A를 **왼쪽 or 오른쪽으로 밀면**, 손님은 1번으로 입장이 가능합니다.

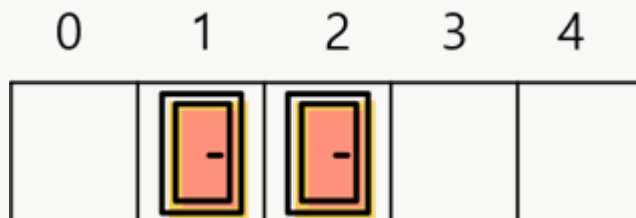
예약고객들이 입장하는 순서는 정해져있습니다.

빠른 고객입장을 위해, 최소 횟수로 미닫이 문 A, B를 밀어야합니다.

[문을 이동하는 방법]

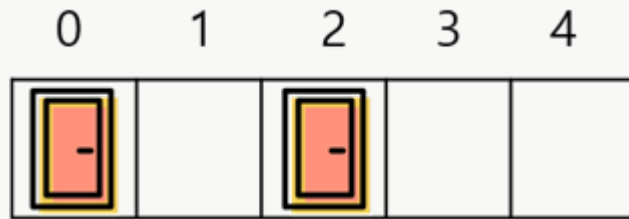
미닫이문의 초기 상태가 0 1 1 0 0 이고

고객이 1 2 0 순서대로 입장한다면 아래와 같이 미닫이문을 이동시킬 수 있습니다.



여기서 고객이 **1번**으로 들어 온다고 한다면 1번 칸에 있는 A문을 치워줘야 합니다.

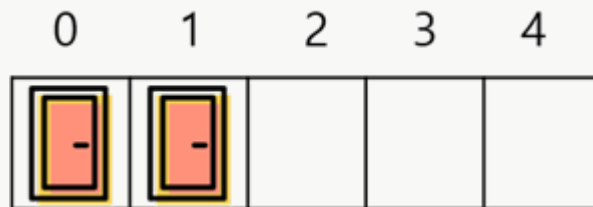
(총 1회 문 이동)



만약 고객이 2번으로 들어 온다고 한다면 B문을 왼쪽 or 오른쪽으로 옮길 수 있습니다.

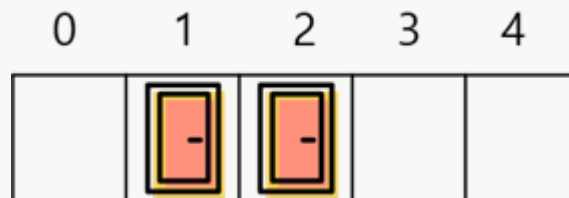
왼쪽으로 옮깁니다.

(총 2회 문 이동)



만약 고객이 0번으로 들어 온다면 A와 B를 각각 한 칸씩 밀어줘야 합니다.

(총 4회 문 이동)



이 예제에서는 총 4회 문을 이동 시켰습니다. 이 값은 최소 문 이동횟수는 아닙니다.

참고로, 위 예제에서 최소 이동횟수는 3회입니다.

[입력]

미닫이 문의 위치를 입력 받으세요.

[출력]

미닫이 문을 이동해야하는 최소 횟수를 출력 해주세요.

[예제]

손님 입장 순서 (하드코딩) : 0 1 0 1 0 1 2 3 2 3 2 3

문의 위치 입력 : 01100

01100 (시작) : 0 번째 손님 입장 완료

01010 (총 1회 이동)

00110 (총 2회 이동) : 1 ~ 5 번째 손님 입장 완료

00101 (총 3회 이동)

01001 (총 4회 이동) : 6 ~ 11 번째 손님 입장 완료

정답 : 4회

[힌트]

문을 이동시키는 네 가지 선택안

1. A를 왼쪽
2. A를 오른쪽
3. B를 왼쪽
4. B를 오른쪽

- Queue 노드에는 A의 위치, B의 위치, 몇 번 손님까지 처리했는지 이렇게 정보를 가지고 BFS를 돌리면 문제를 해결할 수 있습니다.

입력 예제

01100

출력 결과

4



